🚀 Mini Replit Clone – Microservices Architecture (Full Implementation)

📁 Final Project Structure

```
mini-replit-clone/
│
├── api/
│   ├── src/
│   │   ├── controllers/
│   │   │   └── execute.controller.ts
│   │   ├── middleware/
│   │   │   └── auth.middleware.ts
│   │   ├── routes/
│   │   │   └── execute.routes.ts
│   │   ├── services/
│   │   │   └── redis.service.ts
│   │   ├── config/
│   │   │   └── redis.ts
│   │   ├── server.ts
│   │   └── app.ts
│   ├── package.json
│   └── tsconfig.json
│
├── worker/
│   ├── src/
│   │   ├── docker/
│   │   │   └── docker.service.ts
│   │   ├── queue/
│   │   │   └── consumer.ts
│   │   └── index.ts
│   ├── package.json
│   └── tsconfig.json
│
├── docker-images/
│   └── node/
│       └── Dockerfile
│
└── docker-compose.yml
```

🏗️ FINAL PROJECT STRUCTURE

```
mini-replit-micro/
│
├── docker-compose.yml
├── .env
│
├── shared/
│   ├── db.js
│   ├── jwt.js
│   └── authMiddleware.js
│
├── auth-service/
│
├── project-service/
│
├── execution-service/
│
├── sandbox-worker/
│
├── docker-runtime/
│
└── workspaces/
```

🌍 ROOT FILES

📄 `.env`

```
JWT_SECRET=SUPER_SECRET
DB_USER=repl
DB_PASS=repl
DB_NAME=repl
DB_HOST=postgres
RABBITMQ_URL=amqp://rabbitmq
```

📄 `docker-compose.yml`

```
version: "3.9"

services:
  postgres:
    image: postgres:15
```

```
    environment:
      POSTGRES_USER: repl
      POSTGRES_PASSWORD: repl
      POSTGRES_DB: repl
    ports:
      - "5432:5432"

  rabbitmq:
    image: rabbitmq:3-management
    ports:
      - "5672:5672"
      - "15672:15672"

  auth-service:
    build: ./auth-service
    ports:
      - "3001:3001"
    env_file: .env
    depends_on:
      - postgres

  project-service:
    build: ./project-service
    ports:
      - "3002:3002"
    env_file: .env
    volumes:
      - ./workspaces:/workspaces
    depends_on:
      - postgres

  execution-service:
    build: ./execution-service
    ports:
      - "3003:3003"
      - "4000:4000"
    env_file: .env
    depends_on:
      - rabbitmq

  sandbox-worker:
    build: ./sandbox-worker
    env_file: .env
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - ./workspaces:/workspaces
    depends_on:
      - rabbitmq
```

📦 SHARED MODULES

📄 `shared/db.js`

```
import pg from "pg";

export const pool = new pg.Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
});
```

📄 `shared/jwt.js`

```
import jwt from "jsonwebtoken";

export const signToken = (payload) =>
  jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: "1d" });

export const verifyToken = (token) =>
  jwt.verify(token, process.env.JWT_SECRET);
```

📄 `shared/authMiddleware.js`

```
import { verifyToken } from "./jwt.js";

export function authMiddleware(req, res, next) {
  const header = req.headers.authorization;
  if (!header) return res.status(401).json({ error: "No token" });

  const token = header.split(" ")[1];

  try {
    const decoded = verifyToken(token);
    req.user = decoded;
    next();
  } catch {
    return res.status(401).json({ error: "Invalid token" });
  }
}
```

🔐 AUTH SERVICE

📄 `auth-service/package.json`

```
{
  "type": "module",
  "dependencies": {
    "bcrypt": "^5.1.0",
    "express": "^4.18.2",
    "pg": "^8.11.0",
    "jsonwebtoken": "^9.0.0"
  }
}
```

📄 `auth-service/Dockerfile`

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
CMD ["node", "server.js"]
```

📄 `auth-service/server.js`

```
import express from "express";
import bcrypt from "bcrypt";
import { pool } from "../shared/db.js";
import { signToken } from "../shared/jwt.js";

const app = express();
app.use(express.json());

await pool.query(`
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE TABLE IF NOT EXISTS users(
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  email TEXT UNIQUE,
  password TEXT
);
`);
```

```
app.post("/register", async (req, res) => {
  const { email, password } = req.body;
  const hash = await bcrypt.hash(password, 10);

  const result = await pool.query(
    "INSERT INTO users(email,password) VALUES($1,$2) RETURNING id",
    [email, hash]
  );

  res.json({ userId: result.rows[0].id });
});

app.post("/login", async (req, res) => {
  const { email, password } = req.body;

  const user = await pool.query(
    "SELECT * FROM users WHERE email=$1",
    [email]
  );

  if (!user.rows.length) return res.status(401).send("Invalid");

  const match = await bcrypt.compare(password, user.rows[0].password);
  if (!match) return res.status(401).send("Invalid");

  const token = signToken({ id: user.rows[0].id });
  res.json({ token });
});

app.listen(3001, () => console.log("Auth running"));
```

📁 PROJECT SERVICE

📄 `project-service/package.json`

```
{
  "type": "module",
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.0",
    "uuid": "^9.0.0"
  }
}
```

📄 `project-service/server.js`

```
import express from "express";
import fs from "fs";
import path from "path";
import { v4 as uuid } from "uuid";
import { pool } from "../shared/db.js";
import { authMiddleware } from "../shared/authMiddleware.js";

const app = express();
app.use(express.json());

await pool.query(`
CREATE TABLE IF NOT EXISTS projects(
  id UUID PRIMARY KEY,
  user_id UUID,
  name TEXT,
  status TEXT
);
`);

const WORKSPACE = "/workspaces";

app.post("/create", authMiddleware, async (req, res) => {
  const { name, dependencies } = req.body;
  const id = uuid();
  const projectPath = path.join(WORKSPACE, id);

  fs.mkdirSync(projectPath, { recursive: true });

  fs.writeFileSync(
    path.join(projectPath, "package.json"),
    JSON.stringify({
      name,
      version: "1.0.0",
      type: "module",
      dependencies
    }, null, 2)
  );

  fs.writeFileSync(
    path.join(projectPath, "index.js"),
    `console.log("Project ${name} running");`
  );

  await pool.query(
    "INSERT INTO projects(id,user_id,name,status) VALUES($1,$2,$3,$4)",
    [id, req.user.id, name, "created"]
  );
```

```
  res.json({ projectId: id });
});

app.listen(3002, () => console.log("Project service running"));
```

🚀 EXECUTION SERVICE

📄 `execution-service/package.json`

```
{
  "type": "module",
  "dependencies": {
    "amqplib": "^0.10.3",
    "dockerode": "^3.3.0",
    "express": "^4.18.2",
    "ws": "^8.13.0",
    "jsonwebtoken": "^9.0.0"
  }
}
```

📄 `execution-service/server.js`

```
import express from "express";
import amqp from "amqplib";
import { WebSocketServer } from "ws";
import Docker from "dockerode";
import { authMiddleware } from "../shared/authMiddleware.js";
import { verifyToken } from "../shared/jwt.js";

const app = express();
app.use(express.json());

const docker = new Docker({ socketPath: "/var/run/docker.sock" });

const conn = await amqp.connect(process.env.RABBITMQ_URL);
const channel = await conn.createChannel();
await channel.assertQueue("execution");

app.post("/run/:id", authMiddleware, async (req, res) => {
  channel.sendToQueue("execution",
    Buffer.from(JSON.stringify({ projectId: req.params.id }))
  );
  res.json({ message: "Queued" });
```

```
});

const wss = new WebSocketServer({ port: 4000 });

wss.on("connection", (ws, req) => {
  const params = new URLSearchParams(req.url.split("?")[1]);
  const token = params.get("token");
  const containerId = params.get("id");

  try {
    verifyToken(token);

    const container = docker.getContainer(containerId);

    container.attach({
      stream: true,
      stdin: true,
      stdout: true,
      stderr: true
    }).then(stream => {
      ws.on("message", msg => stream.write(msg));
      stream.on("data", chunk => ws.send(chunk.toString()));
    });

  } catch {
    ws.close();
  }
});

app.listen(3003, () => console.log("Execution service running"));
```

🧠 SANDBOX WORKER

📄 `sandbox-worker/package.json`

```
{
  "type": "module",
  "dependencies": {
    "amqplib": "^0.10.3"
  }
}
```

📄 `sandbox-worker/worker.js`

```
import amqp from "amqplib";
import { exec } from "child_process";

const conn = await amqp.connect(process.env.RABBITMQ_URL);
const channel = await conn.createChannel();
await channel.assertQueue("execution");

channel.consume("execution", msg => {
  const { projectId } = JSON.parse(msg.content.toString());

  const command = `
  docker run -d
    --name repl-${projectId}
    -v /workspaces/${projectId}:/workspace
    -p 0:3000
    --memory=256m
    --cpus=0.5
    mini-replit-node
    nix develop --command sh -c "npm install && node index.js"
  `;

  exec(command);
  channel.ack(msg);
});
```

🐳 DOCKER RUNTIME

📄 `docker-runtime/Dockerfile`

```
FROM nixos/nix
WORKDIR /workspace
RUN mkdir -p /etc/nix &&
    echo "experimental-features = nix-command flakes" >> /etc/nix/nix.conf
COPY flake.nix /workspace/
RUN nix develop --command true
EXPOSE 3000
CMD ["nix", "develop"]
```

📄 `docker-runtime/flake.nix`

```
{
  description = "Node Dev";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-23.11";
  };

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs { inherit system; };
    in {
      devShells.${system}.default = pkgs.mkShell {
        buildInputs = [
          pkgs.nodejs_18
          pkgs.nodePackages.npm
        ];
      };
    };
}
```

🚀 RUN EVERYTHING

1️⃣ Build runtime:

```
docker build -t mini-replit-node ./docker-runtime
```

2️⃣ Start system:

```
docker-compose up --build
```

🎯 What You Now Have

✔️ Auth microservice
✔️ JWT middleware
✔️ Project workspace generator
✔️ RabbitMQ event execution
✔️ Docker sandbox worker
✔️ Nix environment

✔️ WebSocket terminal
✔️ Resource limits

This is a real cloud IDE backend similar to:

- Replit
- GitHub Codespaces

If you want next:

- Production-secure hardened version
- Kubernetes version
- Full frontend terminal UI
- Multi-organization support
- Proper Dockerode container management (instead of exec)

Tell me the next evolution you want 🚀

Testing 🚀

We'll test:

1. ✅ Register
2. ✅ Login (get JWT)
3. ✅ Create Project
4. ✅ Run Project
5. ✅ (Optional) Connect WebSocket terminal

Assumption:
You ran:

```
docker build -t mini-replit-node ./docker-runtime
docker-compose up --build
```

Services running:

- Auth → http\://localhost: `3001`
- Project → http\://localhost: `3002`
- Execution → http\://localhost: `3003`
- WebSocket → ws\://localhost: `4000`

🔐 STEP 1 — Register User

Method:

POST

URL:

```
http://localhost:3001/register
```

Body → JSON:

```
{
  "email": "test@example.com",
  "password": "123456"
}
```

Expected Response:

```
{
  "userId": "uuid-here"
}
```

🔑 STEP 2 — Login (Get JWT)

Method:

POST

URL:

```
http://localhost:3001/login
```

Body → JSON:

```
{
  "email": "test@example.com",
  "password": "123456"
}
```

Expected Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Copy this token.

📁 STEP 3 — Create Project

Now we test protected route (JWT required).

Method:

POST

URL:

```
http://localhost:3002/create
```

Headers:

```
Authorization: Bearer <PASTE_TOKEN_HERE>
Content-Type: application/json
```

Body:

```
{
  "name": "my-app",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Expected Response:

```
{
  "projectId": "uuid-project-id"
}
```

Copy `projectId`.

Also check:

```
mini-replit-micro/workspaces/<projectId>/
```

You should see:

- package.json
- index.js

🚀 STEP 4 — Run Project

Method:

POST

URL:

```
http://localhost:3003/run/<projectId>
```

Example:

```
http://localhost:3003/run/3e2d-4a5b-6c7d
```

Headers:

```
Authorization: Bearer <TOKEN>
```

Expected Response:

```
{
  "message": "Queued"
}
```

🐳 STEP 5 — Verify Container Started

Run in terminal:

```
docker ps
```

You should see:

```
repl-<projectId>
```

🌐 STEP 6 — Check If App Running

Your container maps random host port (`-p 0:3000`).

Find port:

```
docker port repl-<projectId>
```

Example output:

```
3000/tcp -> 0.0.0.0:49153
```

Now open browser:

```
http://localhost:49153
```

You should see console output or Express response.

🖥️ STEP 7 — Test WebSocket Terminal (Postman)

Postman now supports WebSocket.

In Postman:

New → WebSocket Request

URL:

```
ws://localhost:4000?token=<TOKEN>&id=repl-<projectId>
```

Example:

```
ws://localhost:4000?token=eyJhbGciOi...&id=repl-3e2d4a
```

Click Connect.

Now type:

```
ls
```

You should see:

```
index.js
package.json
node_modules
```

You now have a live terminal inside container.

🧪 Full Test Flow Summary

```
1. POST /register
2. POST /login
3. POST /create  (with JWT)
4. POST /run/:id (with JWT)
5. docker ps
6. docker port
7. WebSocket connect
```

⚠️ Common Errors & Fixes

❌ `401` Unauthorized

→ Missing `Authorization: Bearer <token>`

❌ ECONNREFUSED RabbitMQ

→ Make sure docker-compose started correctly

❌ Container not appearing

→ Check worker logs:

```
docker logs <sandbox-worker-container-id>
```

🎯 If Everything Works

You now have:

✔️ Microservice auth
✔️ JWT protection
✔️ Project creation
✔️ RabbitMQ event execution
✔️ Docker sandbox
✔️ Nix runtime
✔️ WebSocket terminal

This is real cloud IDE architecture like:

- Replit
- GitHub Codespaces

If you want next, I can show you:

- How to debug logs properly
- How to auto-detect container port
- How to add stop endpoint
- How to add project status endpoint
- How to production harden

Tell me what you want next 🚀